CS & IT ENGINEERING

COMPUTER ORGANIZATION AND ARCHITECTURE

Instruction & Addressing Modes

Lecture No.- 04

By- Vishvadeep Gothi sir

Recap of Previous Lecture

Topic Multiple Instruction Support

Topic Variable Size Instructions

Topic Instruction Construction for CPU

Topics to be Covered

Topic Instruction Cycle

Topic Fetch & Execution Cycle

Topic Branch Instruction

[NAT]

Ans = $\underline{\underline{2}}$

#Q. Consider a register-memory architecture system (2-address instructions). For this system the following intermediate code is going to be converted in machine code. Minimum how many registers are required in system so that the code can run without register spill?

Consider first operand is always the register operand and it's the destination for operation too.

t1 = X + Y

t2 = t1 – Z

t3 = t1 + t2

t4 = M + t3

Assume X, Y, Z and M are memory operands

$R1 \leftarrow X$

$R1 \leftarrow R1 + Y$

$R2 \leftarrow R1$

$R1 \leftarrow R1 - Z$

$R2 \leftarrow R2 + R1$

$R1 \leftarrow M$

$R1 \leftarrow R1 + R2$

[NAT]

H.W.

#Q. Consider a register-based architecture system (2-address instructions). For this system the following intermediate code is going to be converted in machine code. Minimum how many registers are required in system so that the code can run without register spill?

Consider first operand is always the register operand and it's the destination for operation too.

$t1 = X + Y$

$t2 = t1 - Z$

$t3 = t1 + t2$

$t4 = M + t3$

Assume X, Y, Z and M are memory operands

Topic : Effective Address

- Address of operand in a computation-type instruction or
- The target address in a branch-type instruction.

Topic : Branch Instruction

Assume CPU is currently executing instⁿ I2.

PC = 504

CPU decodes I2 as branch instruction

Condition false

Branch not taken

⇓

Next instⁿ in the sequence should execute next. ⇒ which is I3.

⇓

PC remains 504

Condition True

Branch taken

⇓

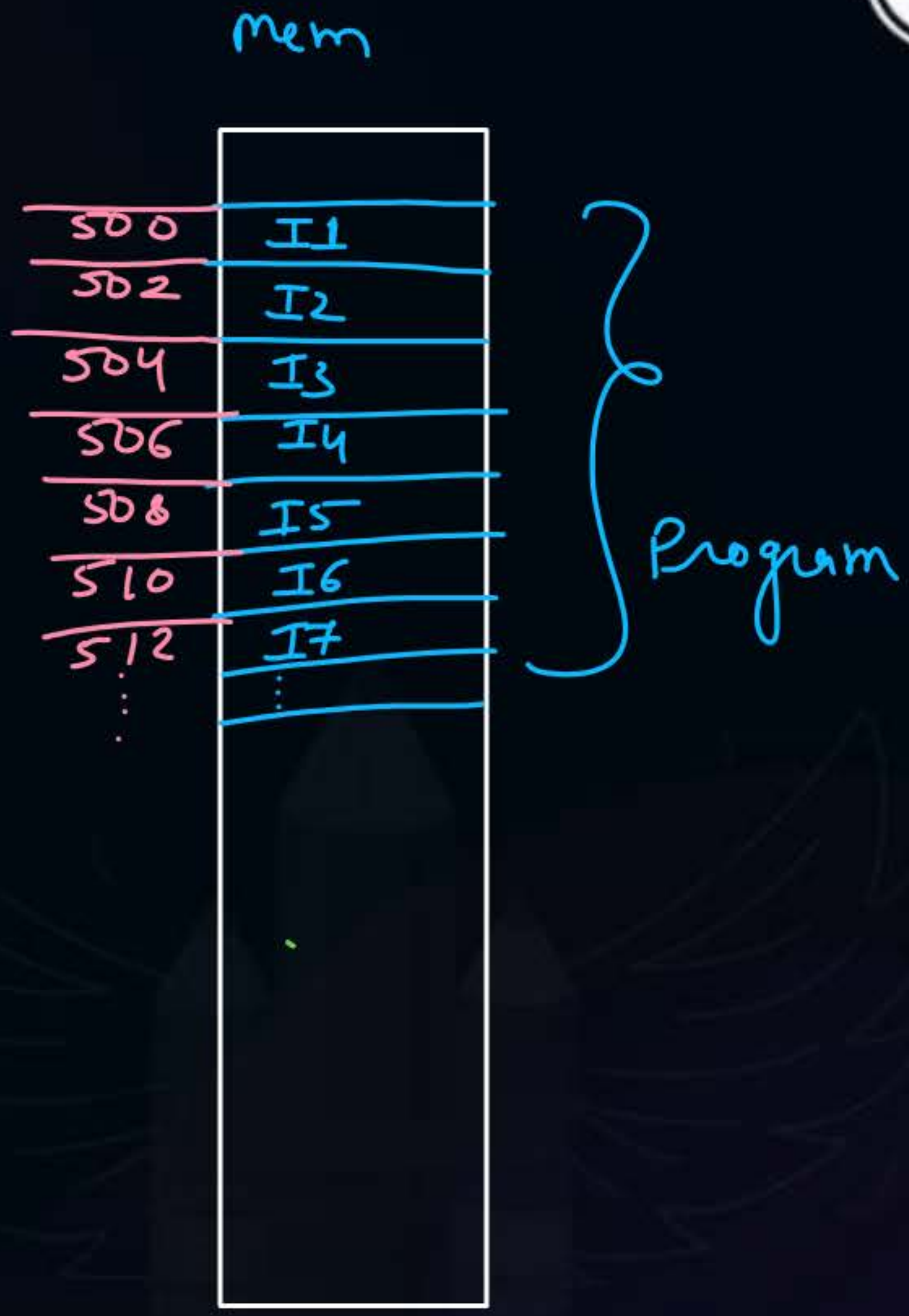
Target instruction of I2 should execute next

⇒ Assume target of I2 is I7

add. of I7 ⇒ Target add. of branch

⇓

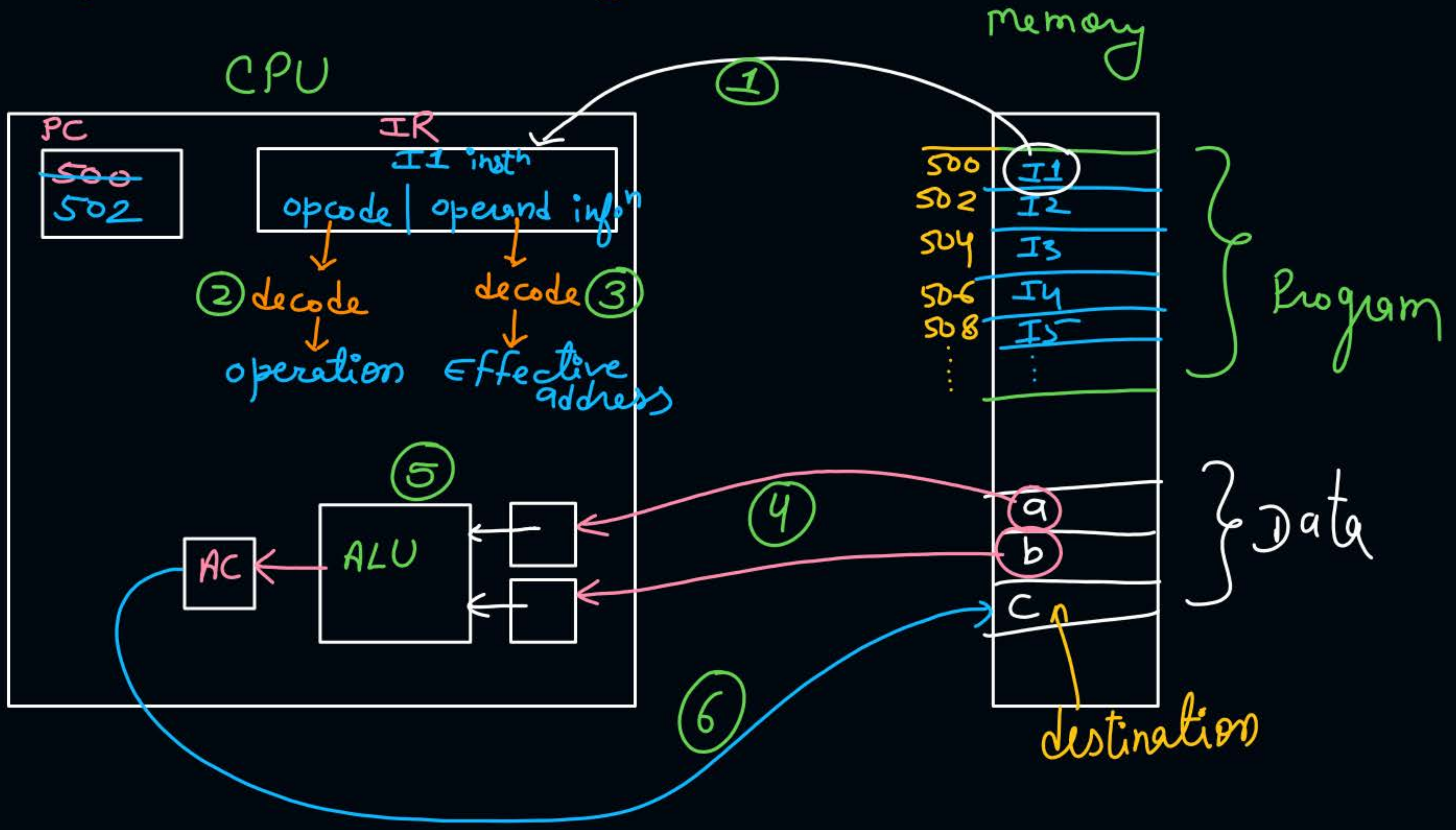
PC = 512



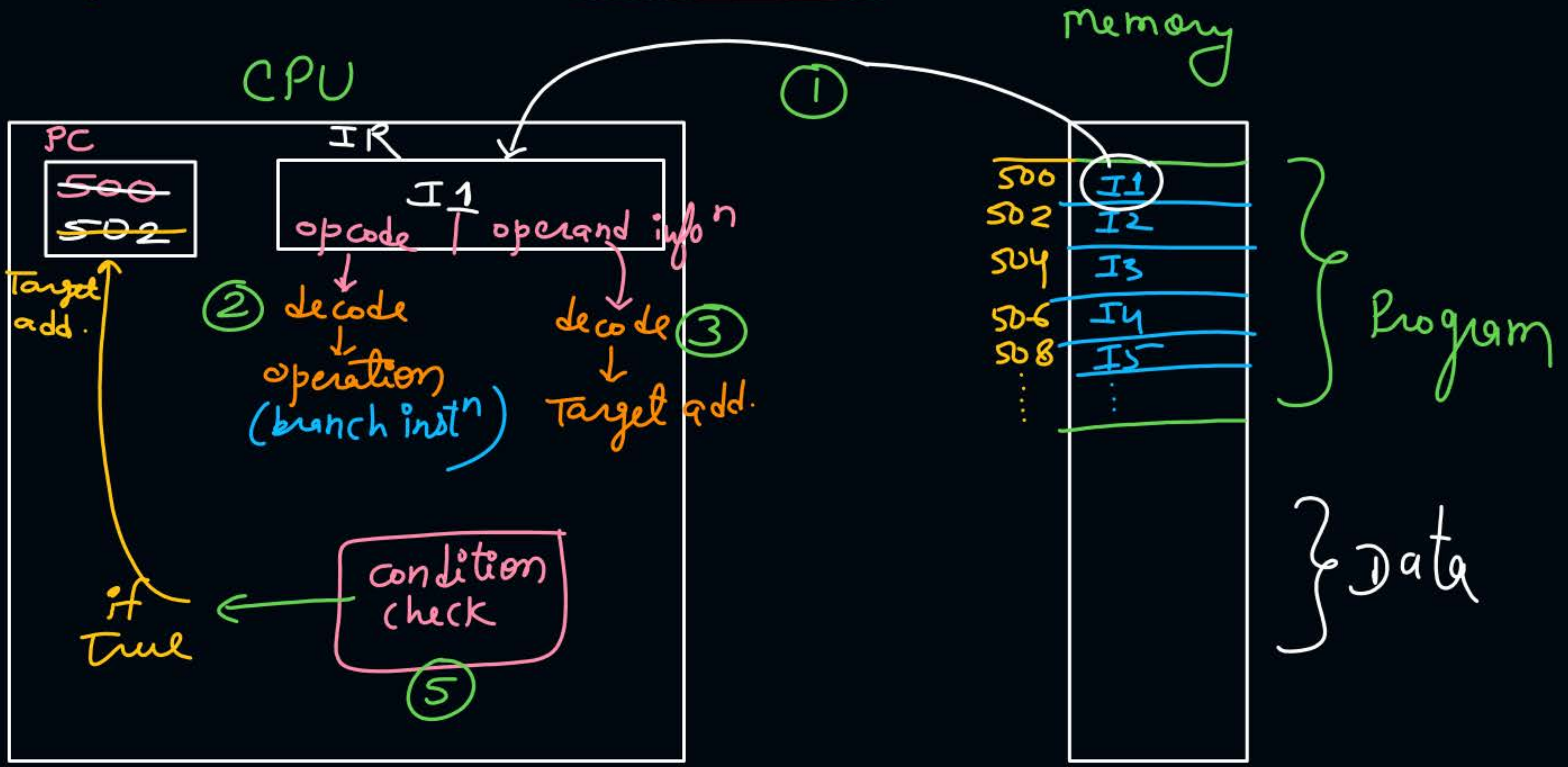
Topic : Instruction Cycle

1. Instruction Fetch
2. Instruction Decode
3. Effective Address Calculation
4. Operand Fetch
5. Execution
6. Write Back Result

Instⁿ cycle for computation type instⁿ



Instⁿ cycle for branch type instⁿ :-



Topic : Instruction Cycle for Branch Instruction

1.) Inst^n fetch :- Inst^n Copied from mem. to IR.
PC increaments

2) Inst^n decode :- CPU decodes opcode and understands operat^n.

3) E.A. Calculation :- CPU calculates target add.

4) operand fetch —

5) Execution ⇒ CPU checks condition, if it is true then PC updated by calculated target add.

6) write back ⇒ —

Topic : Fetch Cycle & Execution Cycle

$Inst^n$ fetch

decode

to

write back

Topic : Why Addressing Modes

opcode	address
Load to AC	3

$$\Downarrow$$

$AC \leftarrow \#3$

or

$AC \leftarrow R3$

or

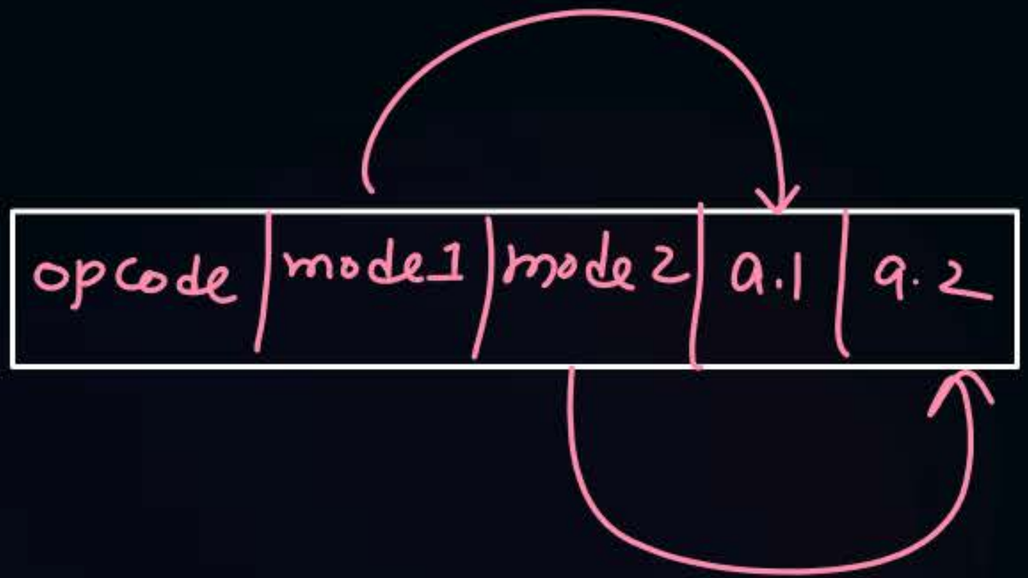
$AC \leftarrow M[3]$

$\Longrightarrow$ Sol^n

opcode	mode	address

Topic : Addressing Modes

- It specifies how and from where the operands are obtained for an instruction



Addressing modes helps to get either operand or effective address.

2 mins Summary

Topic Instruction Cycle

Topic Fetch & Execution Cycle

Topic Branch Instruction

Happy Learning

THANK - YOU